OpenProject — Mitsue Village Project

OpenProject is hosted on a VPS and accessible from any browser at **https://openproject.mitsue.it**. A local instance also runs on Rob's laptop at http://localhost:8080 for development/backup purposes.

Access

URL (VPS — primary)	https://openproject.mitsue.it
URL (local — Rob's laptop)	http://localhost:8080
English Project	Mitsue ai data center (`mistue-ai-data-center`)
Japanese Project	御杖AIデータセンタープロジェクト (`mitsue-jp`)

API Tokens

Each agent has its own account and API token. Use `apikey` as the username.

Agent	Login	API Token
Claude (admin)	`admin`	`d7df8157865c3e15a0c08e6f856afcceadc7c15709684a29766976c82e742c75`
OpenClaw	`openclaw`	`7c73bd59a862df8d99e9981c08966d78200f1b144fc6b6f40a4b54cda8fbed16`
Hermes	`hermes`	`227f76125c000498897183e4d6a11677c45d199807c1f805364aa669f1d354c5`

Making API Calls

```
# List all work packages in the project (VPS)
curl -u "apikey:<TOKEN>" https://openproject.mitsue.it/api/v3/projects/3/work_packages

# Create a work package
curl -u "apikey:<TOKEN>" -X POST https://openproject.mitsue.it/api/v3/projects/3/work_packages
  -H "Content-Type: application/json" \
  -d '{
    "subject": "Task title",
    "_links": {
      "type":     {"href": "/api/v3/types/1"},
      "status":   {"href": "/api/v3/statuses/1"},
      "priority": {"href": "/api/v3/priorities/8"}
    }
  }'

# Update a work package
curl -u "apikey:<TOKEN>" -X PATCH https://openproject.mitsue.it/api/v3/work_packages/<ID> \
  -H "Content-Type: application/json" \
  -d '{"subject": "Updated title", "lockVersion": <current_lock_version>}'
```

Reference IDs

Types

ID	Name
1	Task
2	Milestone
3	Summary task

Statuses

ID	Name
1	New
7	In progress
12	Closed
13	On hold
14	Rejected

Priorities

ID	Name
7	Low
8	Normal
9	High
10	Immediate

Updating Personal Wiki Pages (rob-personal project)

The OpenProject API does **not** support creating or updating wiki pages in this version. Updates go through the Rails console via SSH instead.

Wiki IDs

Project	Project ID	Wiki ID
rob-personal	6	5
Mitsue ai data center	3	(none yet)

Personal wiki pages

Page	Slug	Local file
榧の種 · Kaya Seed Poem	`fei-nozhong-star-kaya-seed-poem`	`/home/rob/Documents/Mitsue/kaya_seed_poem.md`
Mayor Meeting — Kaya Seed Card	`mayor-meeting-kaya-seed-card`	`/home/rob/Documents/Mitsue/mayor-meeting-card.md`

Update procedure

Step 1 — Edit the local `.md` file (e.g. `kaya_seed_poem.md`).

Step 2 — Copy to VPS and into the container:

```
scp /home/rob/Documents/Mitsue/kaya_seed_poem.md root@80.208.225.44:/tmp/kaya_poem.md
scp /home/rob/Documents/Mitsue/mayor-meeting-card.md root@80.208.225.44:/tmp/mayor_card.md
ssh root@80.208.225.44 "docker cp /tmp/kaya_poem.md openproject-web-1:/tmp/kaya_poem.md"
ssh root@80.208.225.44 "docker cp /tmp/mayor_card.md openproject-web-1:/tmp/mayor_card.md"
```

Step 3 — Run the Rails updater:

```
ssh root@80.208.225.44 "docker exec openproject-web-1 bash -c 'cd /app && bundle exec rails ru
wiki = Wiki.find(5)
admin = User.find_by(login: \\\\\"admin\\\\\")

page1 = wiki.find_or_new_page(\\\\\"fei-nozhong-star-kaya-seed-poem\\\\\")
page1.text = File.read(\\\\\"/tmp/kaya_poem.md\\\\\")
page1.author = admin
page1.save!
puts \\\\\"Updated: #{page1.title}\\\\\"

page2 = wiki.find_or_new_page(\\\\\"mayor-meeting-kaya-seed-card\\\\\")
page2.text = File.read(\\\\\"/tmp/mayor_card.md\\\\\")
page2.author = admin
page2.save!
puts \\\\\"Updated: #{page2.title}\\\\\"
\"'"
```

View in browser:

- https://openproject.mitsue.it/projects/rob-personal/wiki/fei-nozhong-star-kaya-seed-poem
- https://openproject.mitsue.it/projects/rob-personal/wiki/mayor-meeting-kaya-seed-card

Adding a new wiki page

```
# On the VPS, inside the container:
wiki = Wiki.find(5)
page = wiki.find_or_new_page("your-slug")
page.title = "Your Title"
page.text = File.read("/tmp/your_file.md")
page.author = User.find_by(login: "admin")
page.save!
```

Project Structure (imported from mitsue_todo.xlsx)

Work packages are organised as:

- **Master To-Do** — Summary tasks per phase (Phase 0–4, Cross-cutting), with Tasks nested as children
- **Legal Checklist** — Summary tasks per category (Entity & Tax, Permits, Pro support), with Tasks as children
- **Risk Register** — Flat list of Tasks prefixed `[Risk]`

VPS Setup (openproject.mitsue.it)

Infrastructure

- **VPS:** 80.208.225.44 (Ubuntu 24.04, 8GB RAM, 3 vCPU)
- **Provider:** Contabo (shared with other yr-design.biz sites)
- **Web server:** Apache2 (managed by Virtualmin)

- **SSL:** Let's Encrypt via Virtualmin, auto-renews
- **Docker files:** `/opt/openproject/`

Architecture

OpenProject runs in Docker on port 8080 (internal only). Apache acts as a reverse proxy, handling SSL and forwarding traffic to Docker.

```
Browser → Apache :443 (SSL) → Docker OpenProject :8080
```

Docker Management (VPS)

```
ssh root@80.208.225.44

# Status
cd /opt/openproject && docker compose ps

# Logs
docker compose logs -f web

# Restart
docker compose restart web

# Stop all
docker compose down

# Start all
docker compose up -d
```

Patched Image

The official `openproject/openproject:17.5.0` image is patched to unlock Gantt PDF export (enterprise feature). The patch is defined in `/opt/openproject/Dockerfile`:

```
FROM openproject/openproject:17.5.0

RUN set -e \
 && sed -i 's/EnterpriseToken.allows_to?(:gantt_pdf_export)/true/' \
        /app/app/components/work_packages/exports/pdf/export_settings_component.rb \
 && if grep -q 'EnterpriseToken.allows_to?(:gantt_pdf_export)' \
        /app/app/components/work_packages/exports/pdf/export_settings_component.rb; then \
      echo 'PATCH FAILED: gantt patch 1 did not fully apply'; exit 1; \
    fi \
 && sed -i 's/render_403 unless EnterpriseToken.allows_to?(:gantt_pdf_export)/# gantt export a
        /app/app/helpers/work_packages_controller_helper.rb \
 && if grep -q 'render_403 unless EnterpriseToken.allows_to?(:gantt_pdf_export)' \
        /app/app/helpers/work_packages_controller_helper.rb; then \
      echo 'PATCH FAILED: gantt patch 2 did not fully apply'; exit 1; \
    fi
```

The `if grep -q` checks make the build **fail loudly** if a patch doesn't apply (e.g. after an upstream code change), instead of silently deploying a broken image.

To rebuild after an OpenProject version upgrade:

```
ssh root@80.208.225.44
cd /opt/openproject
# Update version tag in Dockerfile (FROM line), then:
docker build -t openproject-patched:17 .
docker compose up -d
```

SECRET_KEY_BASE (required from v17)

v17 enforces a `SECRET_KEY_BASE` environment variable. It is set in `docker-compose.yml` under `x-op-app.environment`. If missing, all app containers exit on startup with an "INSECURE SECRET_KEY_BASE DETECTED" error.

Generate a new value with: `openssl rand -hex 64`

Boot Behavior

OpenProject starts automatically 2 minutes after VPS reboot (via systemd) to avoid competing with Apache and other services during boot.

```
# Check service status
systemctl status openproject

# Manual start/stop
systemctl start openproject
systemctl stop openproject
```

Reset Admin Password

If locked out of the admin account:

```
ssh root@80.208.225.44
docker exec openproject-web-1 bash -c "cd /app && bundle exec rails runner \
  \"u = User.find_by(login: 'admin'); u.failed_login_count = 0; u.password = 'NewPassword!'; u
```

Local Instance (Rob's Laptop)

Docker Management

```
# Start
docker compose -f ~/openproject/docker-compose.yml up -d

# Stop
docker compose -f ~/openproject/docker-compose.yml down

# Logs
docker compose -f ~/openproject/docker-compose.yml logs -f web

# Status
docker compose -f ~/openproject/docker-compose.yml ps
```

Access at http://localhost:8080.

Backup & Restore

Backup (run on laptop)

```
bash "/home/rob/Documents/Mitsue/Mitsue Village Project AI data center/openproject_backup.sh"
```

This will:

1. Export all work packages to `openproject_backup.json` (human-readable)
2. Dump the full PostgreSQL database to `openproject_backup.sql` (restorable)
3. Commit and push both files to Codeberg and GitHub

Restore to VPS

```
# 1. Copy fresh backup to VPS
scp openproject_backup.sql root@80.208.225.44:/opt/openproject/

# 2. On the VPS — stop app containers, restore DB, restart
ssh root@80.208.225.44 "cd /opt/openproject && \
  docker compose stop web worker cron seeder proxy && \
  docker exec openproject-db-1 psql -U postgres -c 'DROP DATABASE openproject' && \
  docker exec openproject-db-1 psql -U postgres -c 'CREATE DATABASE openproject' && \
  docker exec -i openproject-db-1 psql -U postgres openproject < /opt/openproject/openproject_
  docker compose up -d"

# 3. Reset admin password after restore (backup may contain old password)
# See "Reset Admin Password" section above
```

Theme — Robouden Dark

The header and sidebar use robouden dark colors, set via OpenProject's 7 built-in CSS variables:

Variable	Color
primary-button-color	`#3655b5`
accent-color	`#6796e6`
header-bg-color	`#353535`
header-item-bg-hover-color	`#505050`
main-menu-bg-color	`#353535`
main-menu-bg-selected-background	`#505050`
main-menu-bg-hover-background	`#444444`

Generating PDFs from Markdown

Project documents are stored as `.md` files and committed alongside matching `.pdf` files. Regenerate PDFs after editing any `.md` file.

Prerequisites

Tool	Install	Notes
`pandoc`	`sudo apt install pandoc`	Converts Markdown → HTML
`mermaid-filter`	`npm install -g mermaid-filter`	Pandoc filter — renders Mermaid diagrams as PNG images
`google-chrome`	system package	Converts HTML → PDF (headless)

`npx mmdc` (Mermaid CLI) is used internally by `mermaid-filter`; it is installed automatically when `mermaid-filter` runs if not already present.

Single file

```
cd "/home/rob/Documents/Mitsue/Mitsue Village Project AI data center"

# Step 1 — Markdown → HTML (Mermaid diagrams rendered to PNG)
MERMAID_FILTER_FORMAT=png \
  pandoc myfile.md -t html -o /tmp/myfile.html \
  -F /home/rob/.npm-global/bin/mermaid-filter \
  --metadata title="myfile" --standalone

# Step 2 — HTML → PDF via puppeteer (no date/filename headers)
node /tmp/md2pdf.js /tmp/myfile.html myfile.pdf

rm /tmp/myfile.html
```

Why puppeteer instead of `google-chrome --print-to-pdf`? Chrome 112+ new headless ignores `--print-to-pdf-no-header-footer`, so the date/time and filename always appear. The puppeteer script uses the Chrome DevTools Protocol directly with `displayHeaderFooter: false`, which works on all Chrome versions.

Helper script — md2pdf.js

Lives at `/tmp/md2pdf.js` (ephemeral — recreate after reboot):

```
cat > /tmp/md2pdf.js << 'JSEOF'
const puppeteer = require('/home/rob/.npm-global/lib/node_modules/mermaid-filter/node_modules/
const path = require('path');

async function htmlToPdf(htmlFile, pdfFile) {
  const browser = await puppeteer.launch({
    executablePath: '/usr/bin/google-chrome',
    args: ['--no-sandbox', '--disable-gpu'],
    headless: true,
  });
  const page = await browser.newPage();
  await page.goto('file://' + htmlFile, { waitUntil: 'networkidle0' });
  await page.pdf({
    path: pdfFile,
    format: 'A4',
    displayHeaderFooter: false,
    printBackground: true,
    margin: { top: '15mm', bottom: '15mm', left: '15mm', right: '15mm' },
  });
  await browser.close();
}

const [,, htmlFile, pdfFile] = process.argv;
htmlToPdf(path.resolve(htmlFile), path.resolve(pdfFile))
  .then(() => console.log('✓ ' + path.basename(pdfFile)))
  .catch(e => { console.error(e); process.exit(1); });
JSEOF
```

Batch — regenerate all documents

Both `/tmp/md2pdf.js` and `/tmp/gen_pdfs.sh` are ephemeral. Recreate with blocks above then:

```
bash /tmp/gen_pdfs.sh
```

Full batch script:

```
cat > /tmp/gen_pdfs.sh << 'EOF'
#!/bin/bash
set -e
DIR="/home/rob/Documents/Mitsue/Mitsue Village Project AI data center"
FILTER="/home/rob/.npm-global/bin/mermaid-filter"
export MERMAID_FILTER_FORMAT=png
export PUPPETEER_EXECUTABLE_PATH=/usr/bin/google-chrome

files=(
  README.md README_jp.md
  mitsue_email_highreso_intro.md
  mitsue_founding_charter.md
  mitsue_implementation_plan.md mitsue_implementation_plan_jp.md
  mitsue_introduction_a4.md mitsue_introduction_a4_jp.md
  mitsue_mayor_meeting_talking_points.md mitsue_mayor_meeting_talking_points_ja.md
  mitsue_project_founding_story.md mitsue_project_founding_story_jp.md
  mitsue_project_overview_pellegrom.md
  mitsue_qa_briefing.md
  mitsue_village_government_onepager.md mitsue_village_government_onepager_jp.md
  mitsue_wbs.md mitsue_wbs_jp.md
  mitsue_stakeholders.md mitsue_stakeholders_jp.md
)

for md in "${files[@]}"; do
  base="${md%.md}"
  html="/tmp/pdf_build_${base}.html"
  pdf="${DIR}/${base}.pdf"
  echo "→ $md"
  pandoc "${DIR}/${md}" -t html -o "$html" -F "$FILTER" \
    --metadata title="$base" --standalone 2>/dev/null
  node /tmp/md2pdf.js "$html" "$pdf"
  rm -f "$html"
done
echo "Done."
EOF
chmod +x /tmp/gen_pdfs.sh
bash /tmp/gen_pdfs.sh
```

After regenerating, commit and push:

```
cd "/home/rob/Documents/Mitsue/Mitsue Village Project AI data center"
git add *.pdf
git commit -m "docs: regenerate PDFs"
git push origin main && git push github main
```

Notes

- **Mermaid diagrams** (`\`` `mermaid` `blocks) are rendered to PNG by` mermaid-filter `before Chrome converts the HTML to PDF. Files that currently contain Mermaid:` README.md `,` README_jp.md `,` mitsue_stakeholders.md `,` mitsue_stakeholders_jp.md`.
- **Japanese fonts** render correctly because Chrome uses the system font stack; no extra configuration needed.
- **Page size** is Chrome's default (US Letter). The `.md` files use inline `<style>` blocks for font sizes and margins — these are preserved in the HTML→PDF conversion.
- **SVG diagrams** in `mitsue_wbs.md` and `mitsue_wbs_jp.md` are inline HTML and render natively without any filter.

Project Documents (Codeberg)

All documents live at: **https://codeberg.org/YR-Design/mitsue-ai-data-center**

Project Overview & Strategy

Document	View	PDF
Founding Story	view	pdf
Founding Story (Japanese)	view	pdf
Village Government One-Pager	view	pdf
Village Government One-Pager (Japanese)	view	—
Q&A Briefing	view	pdf
Phases & Funding Flowchart	view	pdf

Implementation & Planning

Document	View	PDF
Implementation Plan	view	pdf
Implementation Plan (Japanese)	view	pdf
Mayor Meeting Talking Points	view	—

Legal & Governance

Document	View	PDF
Founding Charter	view	—
Founder Agreement Template	view	pdf

Stakeholders

Document	View
Stakeholders (English)	view
Stakeholders (Japanese)	view
Interactive Graph (English)	view
Interactive Graph (Japanese)	view